



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Design for Reliability

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

Reliable system

A reliable service requires investments in three phases (although prevention is historically prevalent):

- 1) **Prevention**: good design, implementation and testing 
- 2) **Detection** (early is better)
- 3) **Maintenance** and **incident response** capability (faster is better)



Summary: Scalable and reliable architectures

AVOID

- **System bottlenecks**
- **Single points of failure**

through

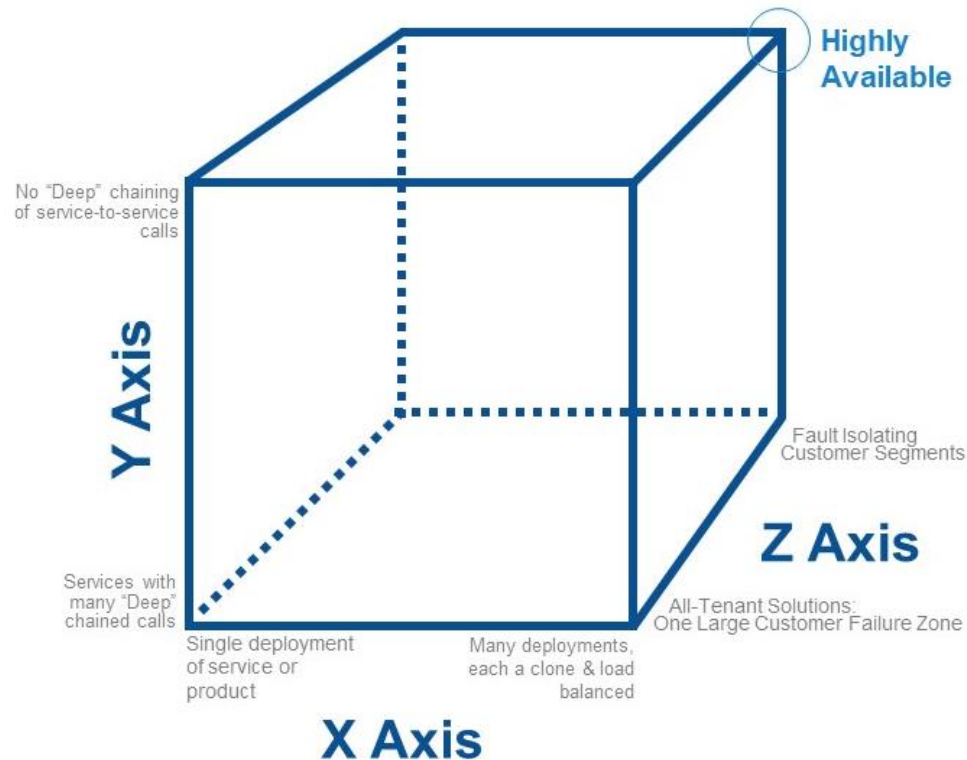
- **Replication at different levels**
- **Fault isolation**



An approach to design through the *Availability cube*

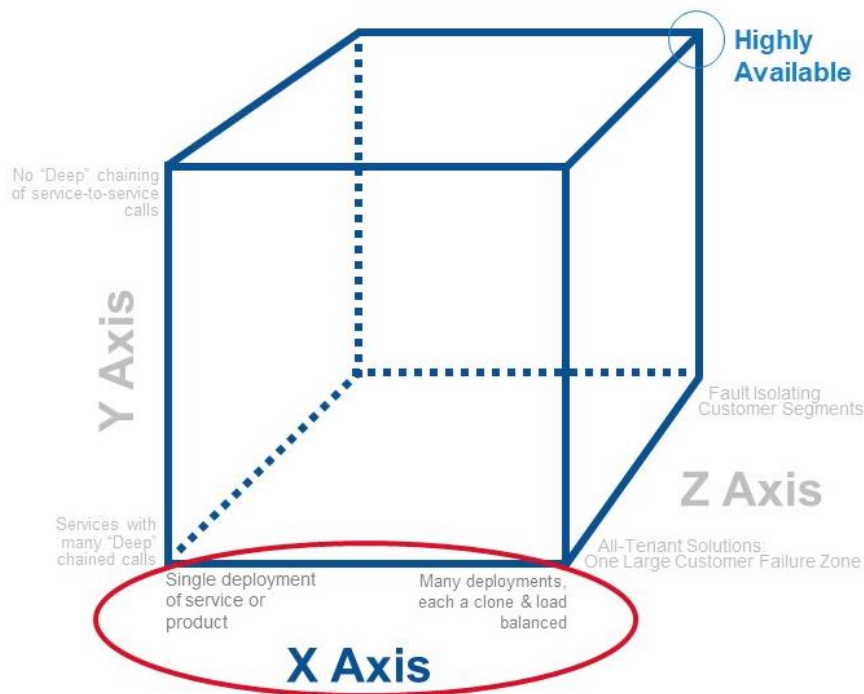
The idea is that any available product/service/solution should have:

- **(X)** multiple deployments
- **(Y)** not deep dependencies among function calls
- **(Z)** fault isolation



X axis of the Availability Cube

- The X axis of the *Scale Cube* indicates cloning/replication for scaling transactions
- The X axis of the *Availability Cube* shows that we can increase overall availability by increasing the number of instances (that is, add extra deployments) of a product/service



- Simple and fast to implement
- Increases availability of independent services
- **Availability is expensive** (*golden rule*)
 - in terms of storage for replicated data sets
 - multiple services require multiple deployments



Pros and Cons

- Better availability
 - Fault isolates customers
 - Fewer noisy neighbors
 - Reduce single points of failure
 - No catastrophic backouts
- Faster releases
 - Lower risk for releases
 - Release to a single segment

The deployment is more difficult: the architecture is complex because more customers segments imply more subsystems that must guarantee high availability and fault isolation



Example

- Let us assume that our application (service/product) has shown a low incidence of failure and low recovery time upon failure. We expect to achieve an availability greater than 99.99% **by adding a second node**
- Adding a second node increases availability because the likelihood that both nodes fail at the same time becomes the probability (1-Availability) that both nodes are down at the same time:

$$1 - ((1 - \text{Availability}_{node1}) \times (1 - \text{Availability}_{node2}))$$

or

$$1 - ((1 - 0.999) \times (1 - 0.999)) = 0.999999$$



Note: Math simplification

- For simplicity reasons, we have used time as an availability metric
- Time-based availability metrics do not show the probability of failure because they indicate a simple relationship between the time a service was available and the total observation time
- “Non-Available” time is really a product of both the probability of failure and the mean time to recover from that failure
 - While strictly mathematically incorrect, we can create a **directionally correct availability evaluation** by using this product as if it were just the probability (because it covers both the probability of a failure AND the average duration of non-availability)



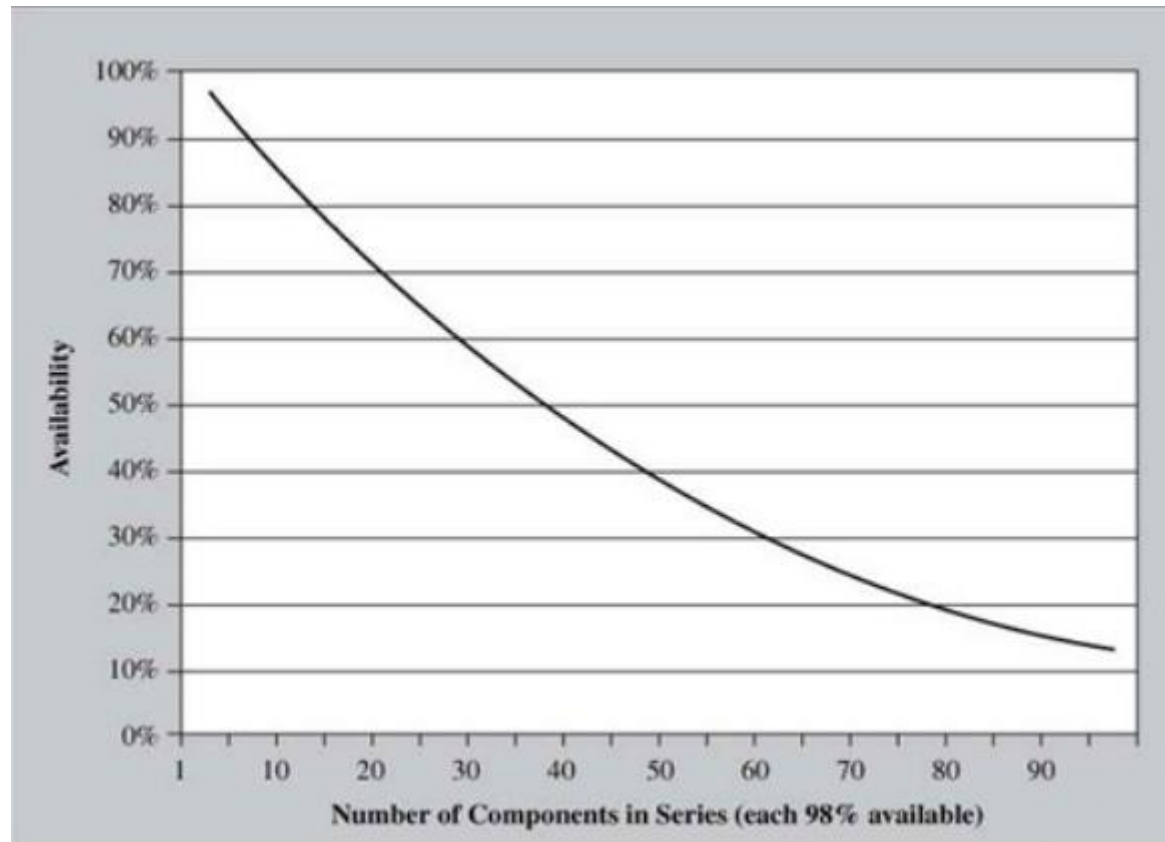
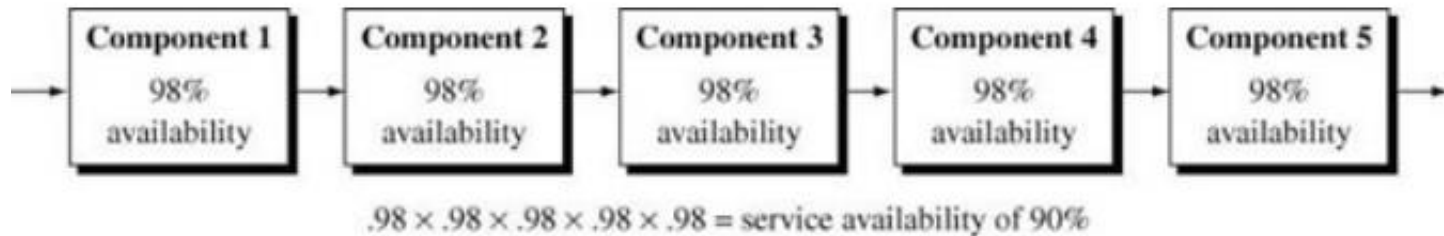
Redundancy

- Redundancy refers to introducing **multiple components** that can be used to carry out the required tasks
- Redundancy is an essential component in any system because it ensures that the system can function properly in the event of an unforeseen issue thus guaranteeing system reliability and more satisfying end-user experience

But how redundant systems can be organized?

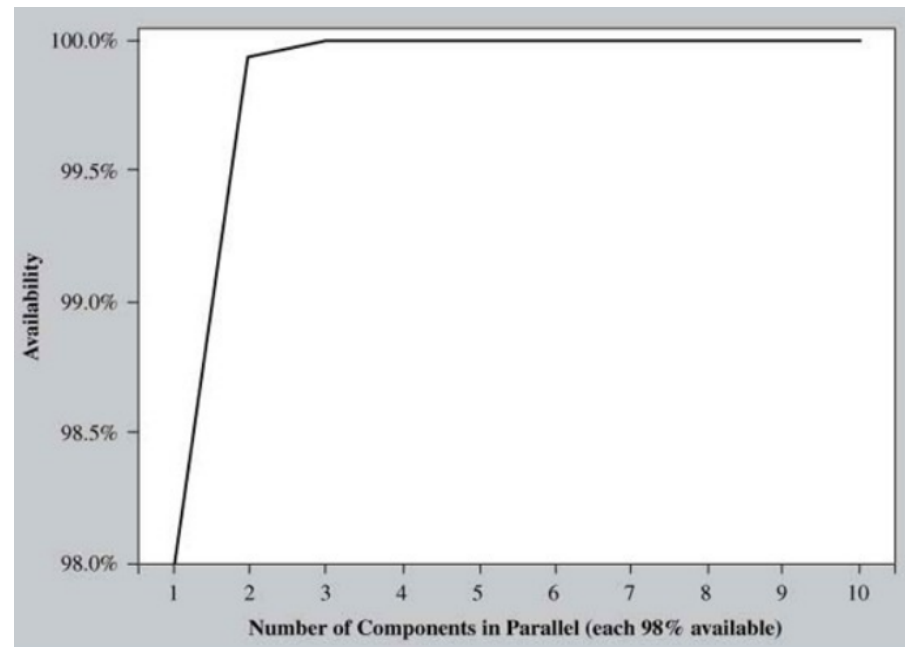
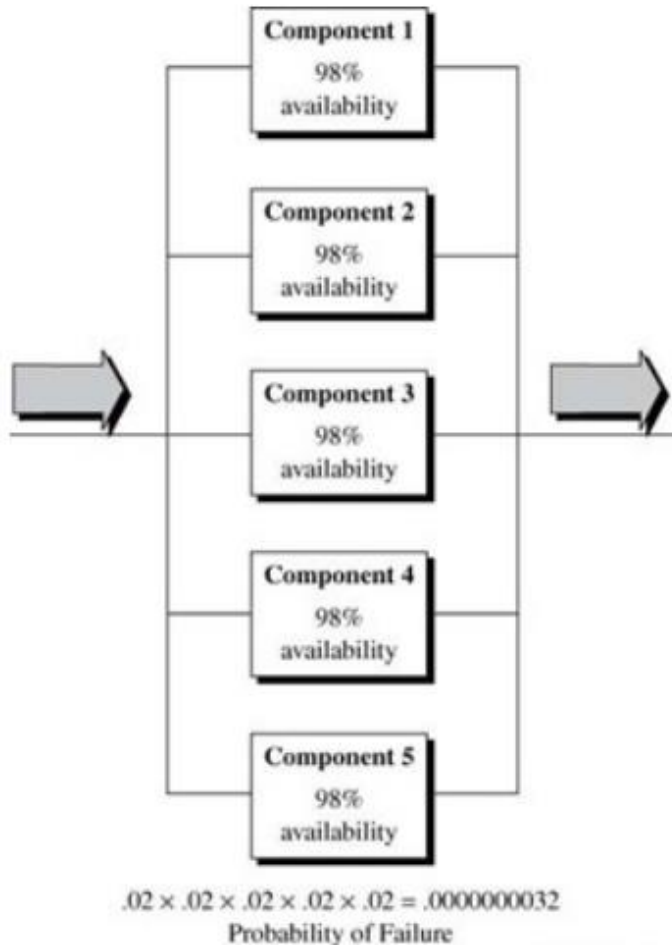


Components in series reduce availability



Components in parallel increase availability

All components must fail in order the service fails



Types of redundancy

There are different kinds of redundancy principles that can be applied to systems, e.g.,

- **N** (*no redundancy*)
- **N+1, N+2, ..., N+X**
- **2N, 2N+1, 2N+2**
- **3N/2**



N+X

- **N+1** denotes the minimum requirement for introducing redundancy to a system. It refers to the base level of resources required for the system functionality plus, for example, a single backup or a replication of one critical service
 - For example, this redundancy level is appropriate for a small IT system, but it is not suited for medium- to large-scale systems
- **N+2** consists of the resources needed for the system functionality plus two separate replications. This increases the redundancy of a system as well as the reliability confidence of end-users about the service
- In addition, there may be instances where even more replications are maintained. They are referred to as **N+X**
 - Unless there is a specific requirement, $X > 2$ is rare



2N

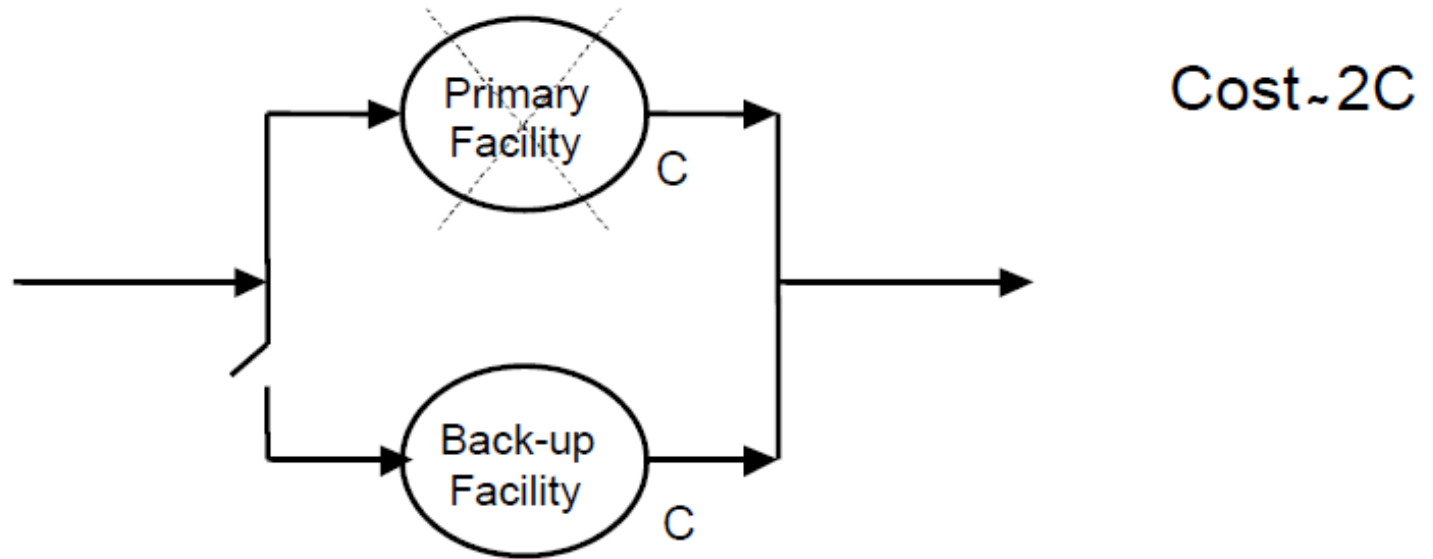
- If N refers to the minimum number of resources required to operate a service, then **2N** (or **N+N**) means that there is twice the amount of required resources/capacity available in the system
- Clearly, **XN** refers to a **X** redundancy of the minimum capacity

Example

- Consider a service in a data center that is based on 10 servers
- A 2N redundancy uses additional 10 servers that act as a dedicated capacity
- Additional questions about redundant servers:
 - **Where?**
 - **How?**



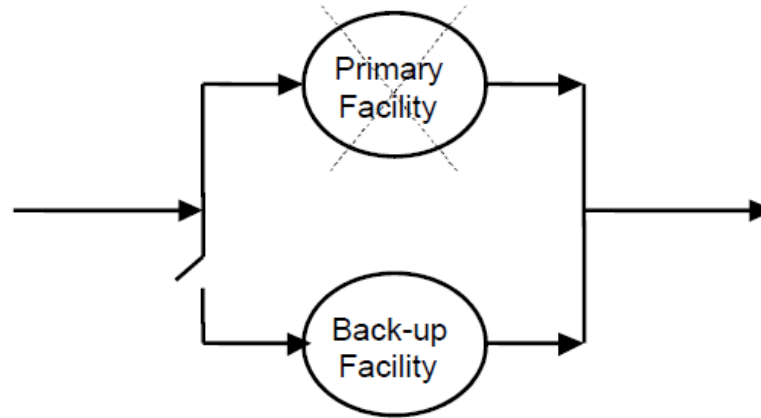
Perfect redundancy



Scenario	Single IT Facility Reliability	Redundant IT Facility Reliability
1	0.90	0.9900
2	0.95	0.9950
3	0.99	0.9999



Key assumptions

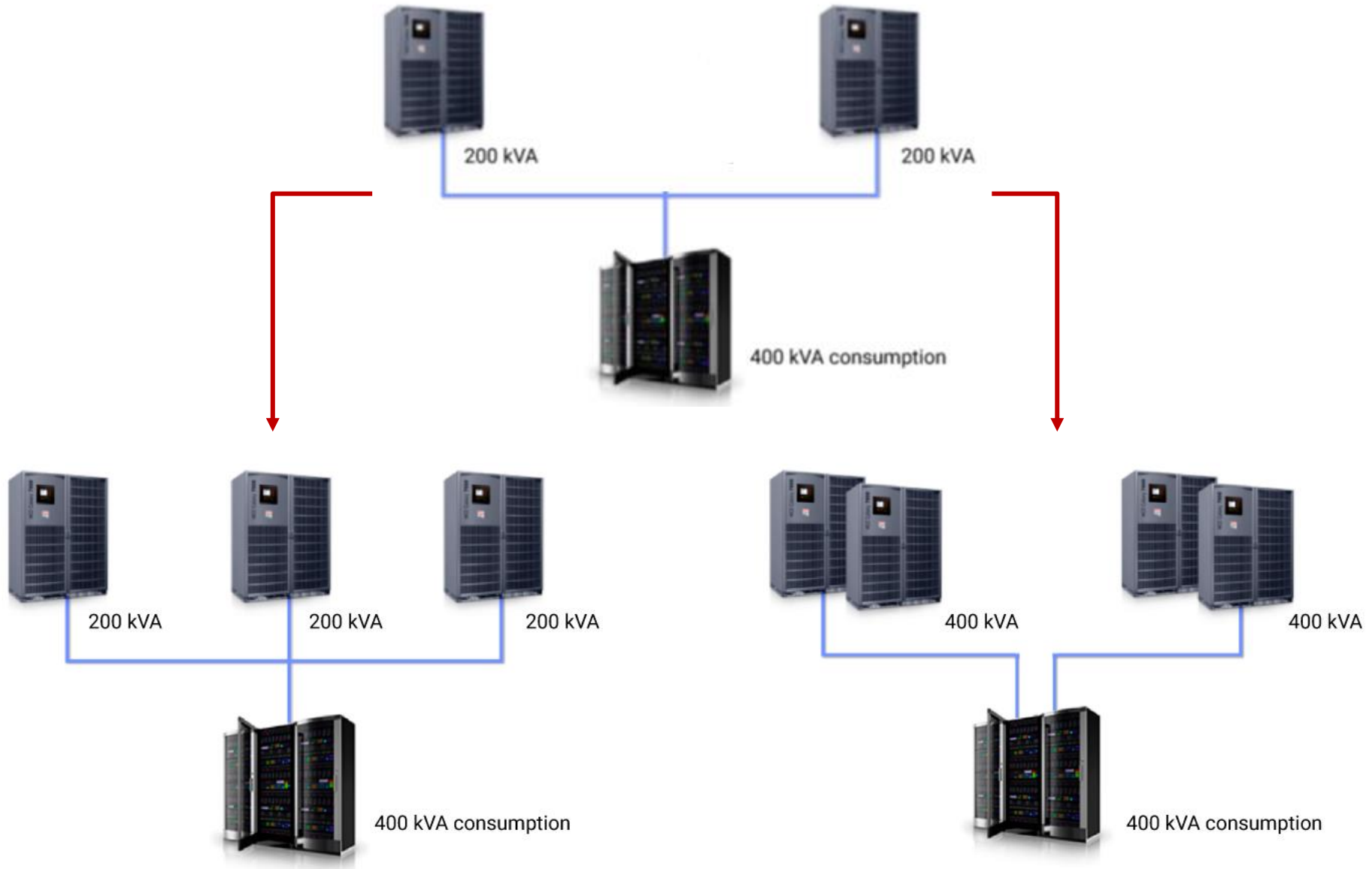


1. Primary and secondary facilities are perfectly aligned
2. Failures are independent
3. Switchover capability is perfect (from the technological and human points of view)

Some organizations violate these assumptions



Examples: N+1 vs 2N



2N+X

- **2N+X** correlates to a system with:
 - **twice** the required resources/capacity to function normally
 - plus **X** replications of a critical component (data, service, etc.) as an additional redundancy step
- For example, 2N+2 refers to a system with twofold additional capacity plus two backups to provide one of the highest levels of redundancy
 - If there are 10 servers in a data center, a 2N+2 system will have another 10 identical servers as a reserved capacity and 2 more servers that will act as backups in case of an emergency

2N+2 is typically considered the highest level of redundancy that is commonly used in the IT industry



3N/2

3N/2 or more generically **AN/B** refers to a redundancy methodology where **additional capacity is based on the load of the system**

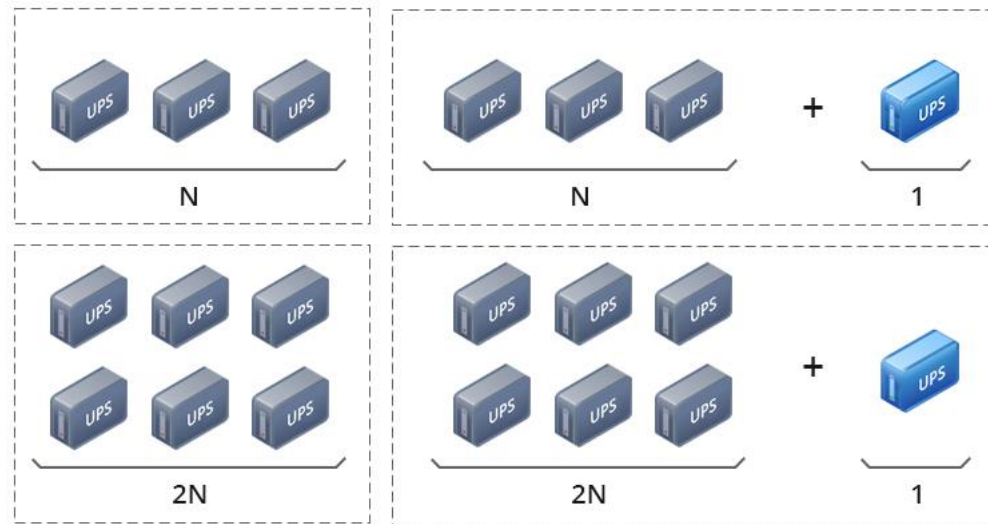
Example (3N/2 redundancy)

- Consider 3N/2 redundancy applied to **power delivery** infrastructure in a data center
- There will be separate power delivery systems powering 2 servers
→ This results in each power delivery system only using 67% of the available capacity at a specific time (or the inverse of the 3N/2 ratio)

3N/2 is a common *rule of thumb* in performance and reliability

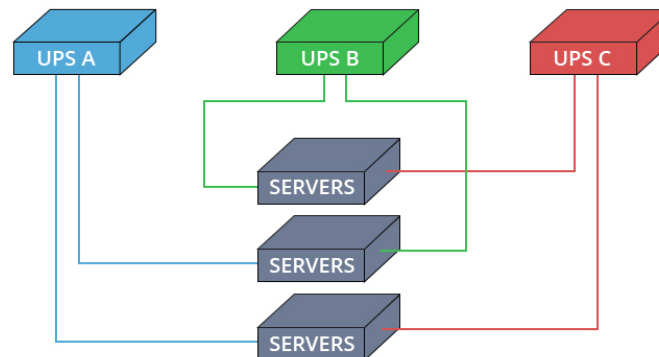


Examples

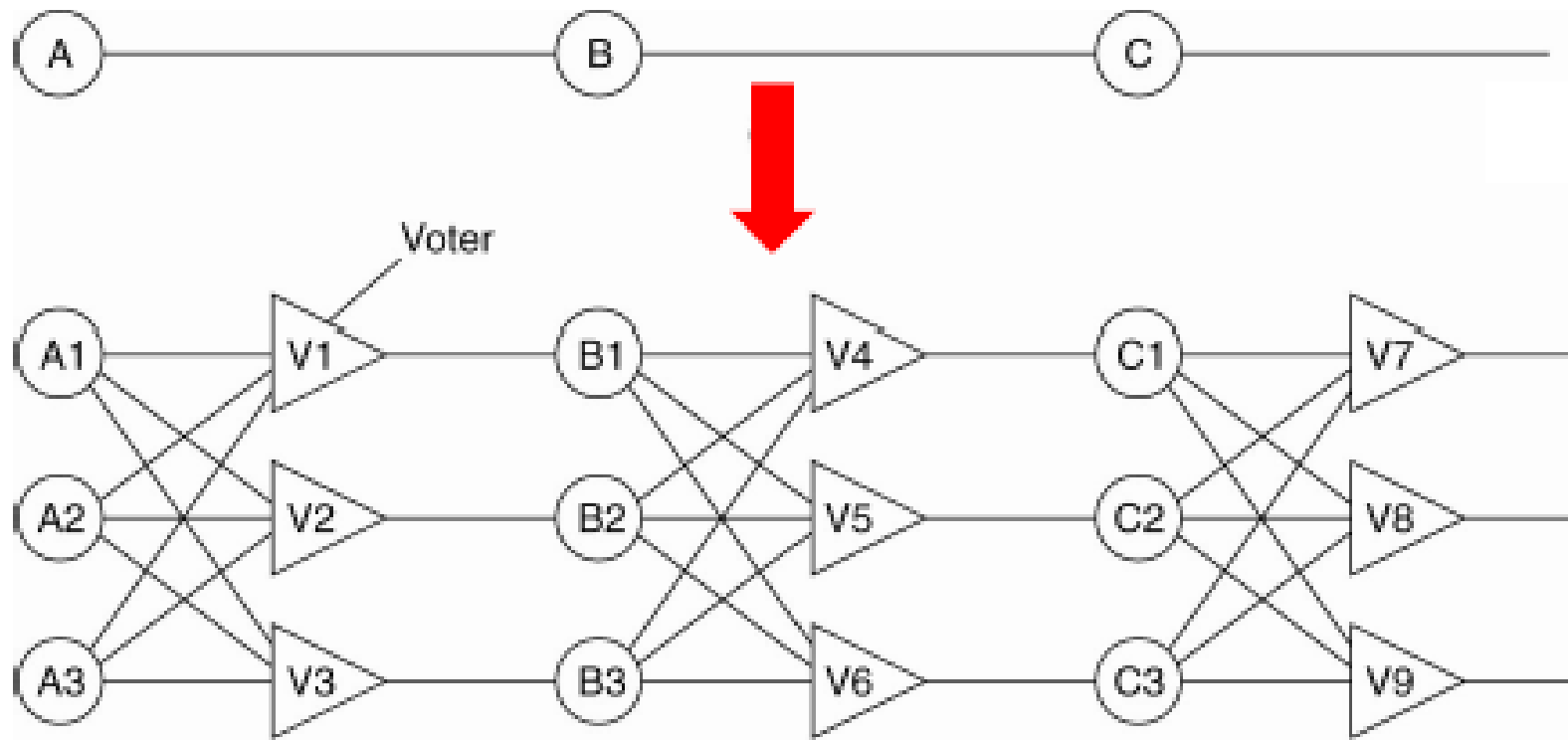


$N=1$ unit: UPS, Generator, or Cooling

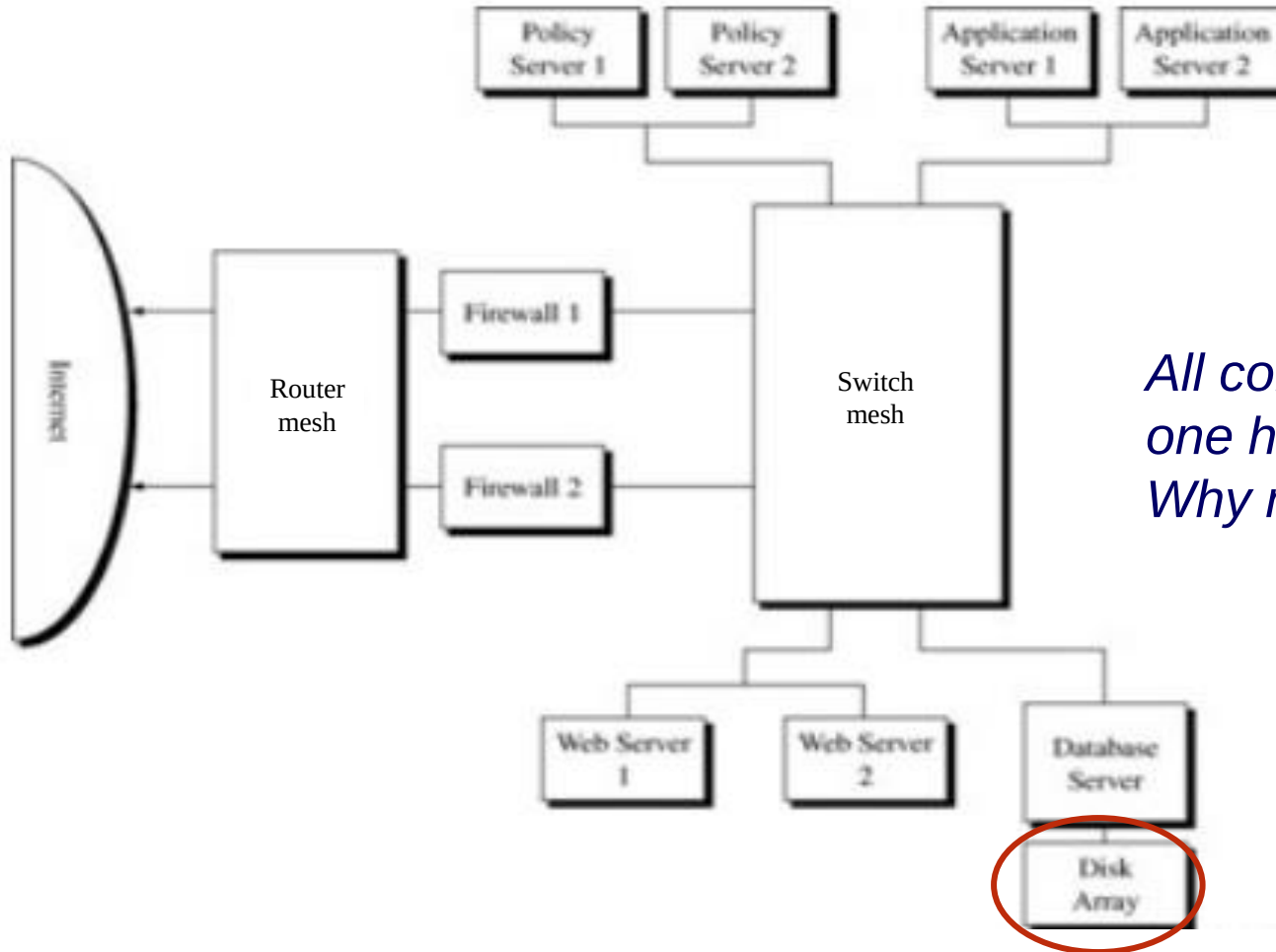
$3N/2$ UPS



Triple modular redundancy (e.g., Air and Space systems)



Example 2N-1



*All components but one have redundancy
Why not all?*

Redundancy: *How to implement it?*

- **Active** (or **hot**): the redundant component is operated simultaneously with the original component. In case the original fails, just the redundant component will be used, and the service is immediately provided at most with some degradation
- **Passive** (or **cold**): the redundant component is available yet not operational while the original component is active. It will be activated –with some delays – to provide functionality in the event of a failure of the primary service
- **Standby** (or **warm**): the redundant component is not operational, but it is ready to operate in a short time period



Replication issues

Using (*active* or *hot*) multiple copies may lead to some performance degradations

- Consistency problems (for data replication cases)
 - Secondary server resources may not be identical to the original system resources in terms of capacity
-
- Moreover, any replication increases:
 - Costs
 - Management complexity



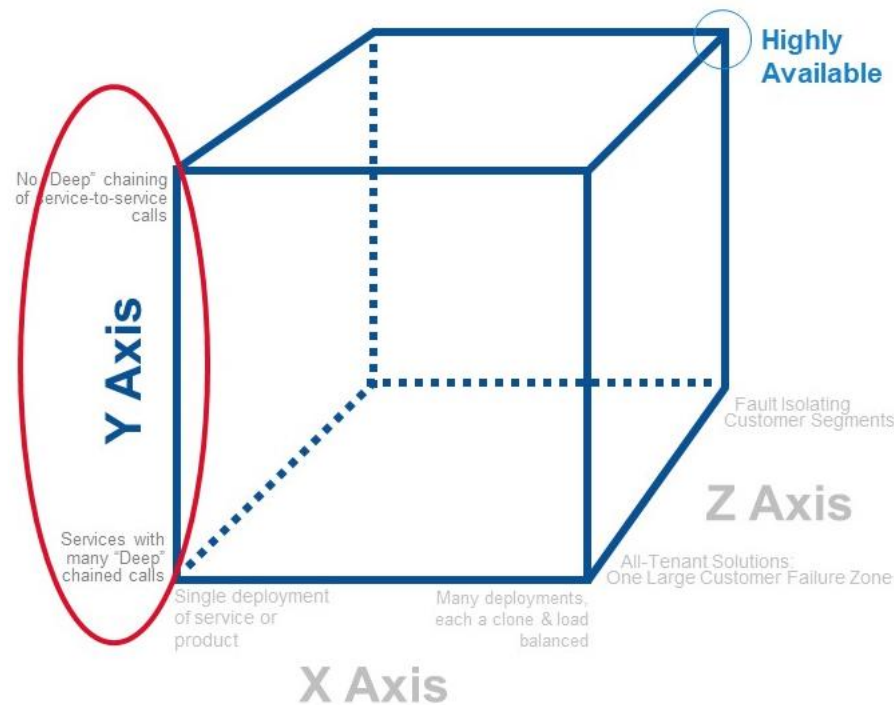
Y axis of the Availability Cube

- The Y axis of the Scale Cube **addresses fault isolation between services**
- Unlike the X of the scale cube, always “more” of something in the Y axis of the Availability Cube is not desirable
 - When it is concerned with the number of service and data as a function of the number of inputs that can be connected to a specified output
 - When service and data fuses
 - Mesh interactions also cause problems
- As such, the Y axis of the cube moves from a large number to near 0 because high degrees of inter-service communication increase the multiplicative effect of failure, thereby reducing availability



Y axis of the Availability Cube

- The Y axis increments the exponent of availability calculation, as applied to the base that is determined by the X axis
- For any connected services, simply count the number of edges in the service-to-service(s) graph and apply that to the base identified in the X axis calculation



Y axis of the Availability Cube

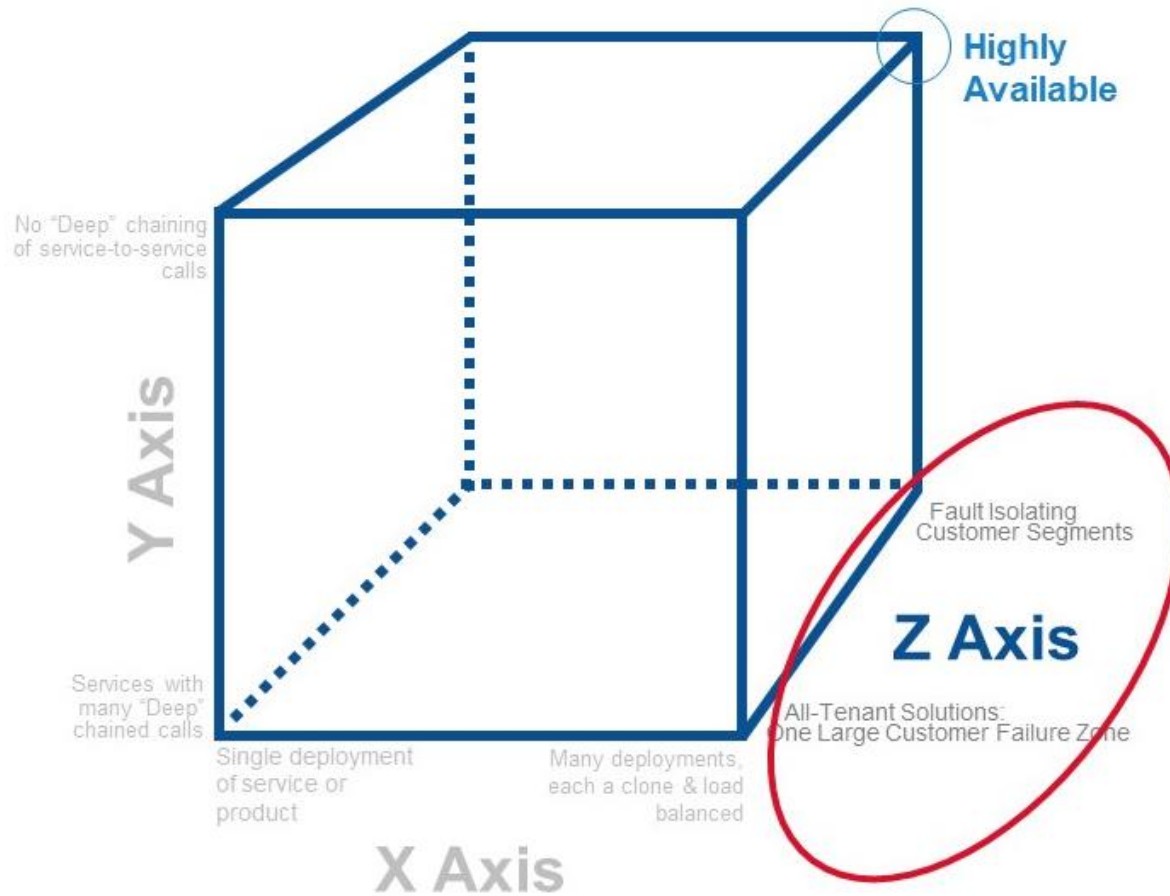
- Anytime we chain synchronously call services (*deep chained calls*), the overall availability is reduced
- If service 1 through service N have a perceived (or theoretical) availability of 99.95%, the availability of the sequence of calls of 1 through N is

$$.9995^N$$

- If $N = 5$, then the availability is reduced to **99.75**
- Hence. We should consider that
 - It reduces availability exponent if it is able to create smaller fault isolation zones
 - For the benefit of higher availability, the architecture is more complex and reduces flexibility of how services can be deployed



Z axis of the Availability Cube



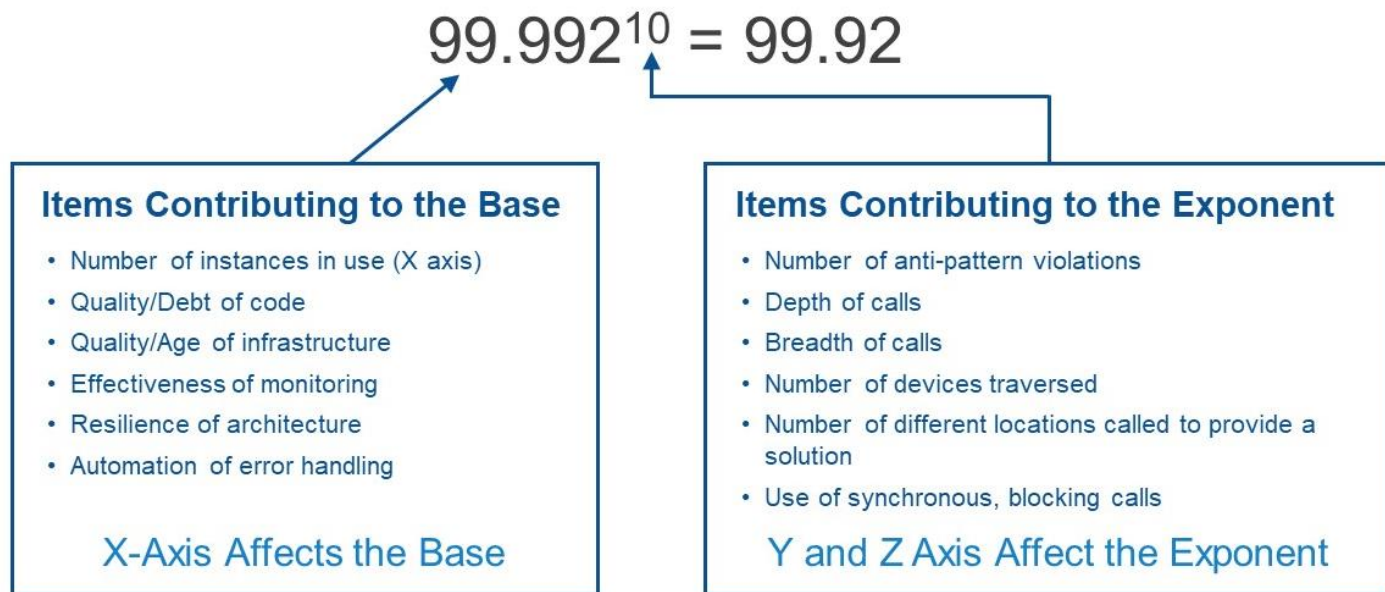
Z axis of the Availability Cube

- Similar to the Z axis of the scalability cube, the Z axis of the Availability Cube defines “blast radius” boundaries (sometimes called *Bulkheads* in microservices or *Swimlanes*)
- Typically, these boundaries are defined by
 - customer boundaries (groups or segments of served customers)
 - geographies of the globe or within a nation
 - possible transaction identifiers within analytics systems
- The boundary assumes that **no synchronous call**, that would otherwise increment the exponent as in the Y axis, **exists between any swim-lane**, bulkhead, or blast containment zone
- As such, **the Z axis serves to constrain the upper bound** for any deployment of the exponent in availability calculations



Contributions of the three axes

- Although we have treated the implied probability of failure and MTTR combination as a probability of failure, we can consider it as “directional” correct. This number works as long as:
 - the failure cause is not significantly increased demand
 - the availability is derived from perceived operations
- This number represents the **base** for a simple exponential equation



Example (1/4)

Let us consider an equities trading application that allows traders to view real-time market price information

- If this information is presented to the user in a browser *application*, then a possible method to deliver this real-time price information is through Web sockets. It is mission critical that the trader **has up to date information**, because a disconnection from the server may result in the trader losing money

Possible architecture

- 1) The server pushes real-time quote data through a Web socket with a maximum time between updates of 3 seconds
- 2) Should that time be exceeded, the browser will contact the server via an HTTP request to determine if everything is okay
- 3) If the browser does not receive a reply indicating the server is in good health, it begins to contact other servers to resume information



Example (2/4)

- To accomplish this failover feat to another server, the browser application must receive a list of servers which can support the application during initialization. (This list is stored in the local memory of the application for future reference)
- Once it is detected that the current server is unavailable, the browser will contact the next server in the list. If the problem arises again, the next server will be selected, and so on until the service is established
- In any case, the browser has to send messages to alert the operations team that a problem exists



Example (3/4)

When the primary server becomes unavailable and the browser application is establishing a connection to the new server, market quote information is not being delivered. This information is mission critical, and it is the responsibility of the application to recover and be able deliver the missing updates to the SPA

- 1) This may be accomplished using a **server-side memory cache** that is replicated to all servers which support the application
- 2) This cache will remember market quote information sent to each instance of the application for a short amount of time (3 minutes). Each entry in the memory cache will have a sequence number and an identifier as to the user who is receiving price quote data
- 3) Each quote message received by the application will contain the sequence number so to remember the last sequence number received

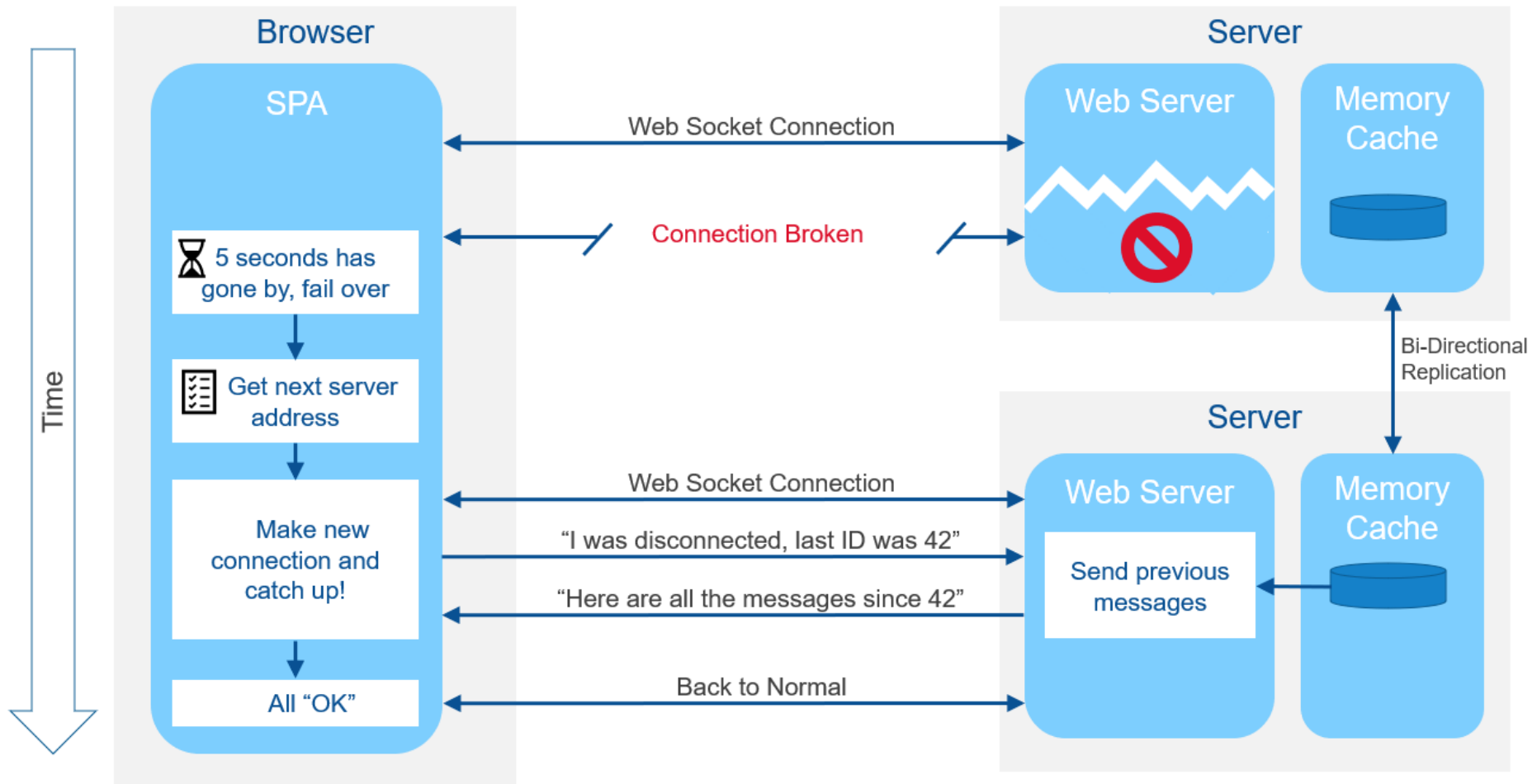


Example (4/4)

- 4) When the application is disconnected from its primary server and establishes a new Web socket connection with the next server in the availability list, it will send a “I was disconnected” message with the username and last sequence number received
- 5) When the server receives that message, it will access its memory cache and send the missing price quote messages in sequence
- 6) Once these have been sent, normal updates over the Web socket connection will resume
- 7) A similar strategy may be implemented within the browser for the case when the browser application and the server application are engaged in a synchronous conversation
- 8) The playback of missing messages is more complex as each side will need to make sure the messages are exchanged in order



Architecture of the example



“It always works”

- From this example, it is possible to understand how to fault isolate a stateful Web socket connection and to ensure the user experience is not affected during a server-side outage of network disconnect
- If we apply all X-Y-Z principles in a core Web application architecture along with other application fault isolation strategies, it is possible to create reliable Web applications which provide users with the desirable experience of **“it always works”**



Fault isolation

A “swim-lane” or **fault isolation zone** is a typical **failure domain**, that is

- a group of services within a boundary such that any failure within that boundary is contained inside
 - so that the failure **does not propagate** or affect services outside of that boundary
- It is important to always think to the “blast radius” of failure and pose the question: “**What other component(s) get(s) impacted when a specific service fails?**”



The term “swim-lanes” depicts the separations, similar to how a floating line of buoys keeps each swimmer in his lane during a race



Two benefits of fault isolation

1. **Fault Detection:** Given a modular approach, the component of availability associated with the time to identify the failure is reduced because the efforts to find the root cause of a failed component can be isolated to the section of the product or platform associated with the failure domain. Once something breaks, it can be more rapidly identified and fixed because the failure is limited in scope. This increases overall availability because **Recovery Time Objectives (RTO)** are decreased
2. **Fault Isolation:** The failure does not propagate and does not cause a deterioration of other services within the platform. As “blast” is contained, depending upon approach, ***only a portion of users or a portion of the service function*** is affected
 - This is similar to electrical circuit breakers in a house: the breaker exists to **limit the fault zone** for any load that exceeds the breaker limit. Failure propagation is contained by the breaker switch preserving power to the other devices which are not affected



Architecture with fault isolation (1/4)

In a fault isolated architecture each **failure domain** is isolated

In order to achieve a similar architecture, user-initiated synchronous calls between fault isolation zones are absolutely forbidden as any call, even with appropriate timeout and detection mechanisms, is likely to cause a **cascading series of blocks and failures across other domains**

You do not have a failure domain if

- 1) that domain is connected via a synchronous call to any other service in another domain
- 2) that domain is connected to any service outside of the domain and can receive synchronous calls from other domains or services



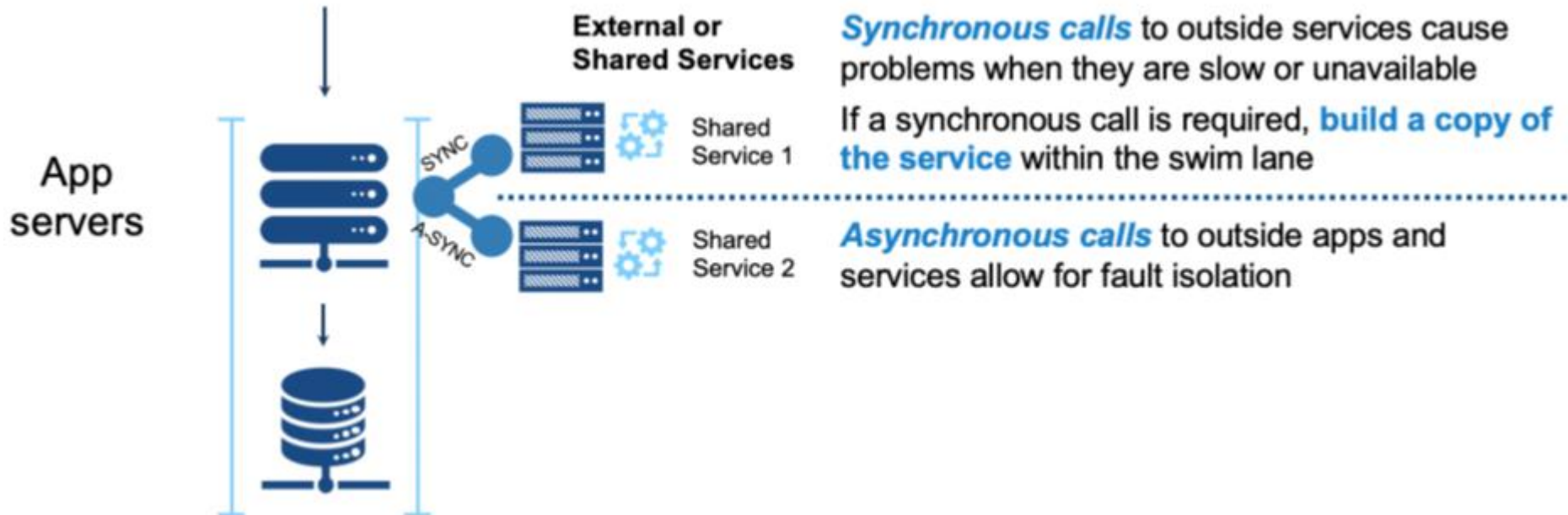
Architecture with isolated faults (2/4)

It is acceptable, but not advisable, to have **asynchronous calls between domains** and to have **non-user initiated synchronous calls between domains** (as in the case of a batch job collecting data for the purposes of reporting in another failure domain)

- If such a communication is **necessary**, it is **very important to include failure detection and timeouts** even with the asynchronous calls to ensure that retries do not call port overloads on any services



Example 1



An app or service is designed for fault isolation when it can "**fire and forget**" or deal with the outside service being slow or unavailable

Architecture with isolated faults (3/4)

A swim-lane should have all its services located within the failure domain

- For instance, if database [read/writes] are necessary, the database with all appropriate information for that swim-lane should exist within the same failure domain as all of the application and Web servers necessary to perform the functions of the swim-lane
- Furthermore, that database should not be used for other requests of service from other swim-lanes →

The golden rule is to use one database on one host



Architecture with isolated faults (4/4)

1. Shared higher level network components (e.g., border systems and core routers) can be rarely isolated
2. Sometimes, if possible, firewalls and load balancers must be isolated. This is necessary under very high demand situations where a single pair of devices would not meet the offered workload
3. The other main components can be properly isolated
 - Web servers
 - Top of rack switches (in non-IaaS implementations)
 - Compute (app servers)
 - Storage



Example 2

1

Edge Network Devices



(Gray icon denotes deployment in redundant pairs)

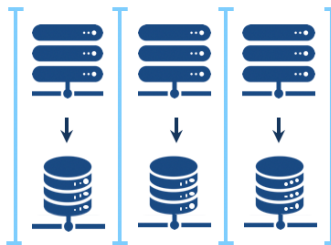
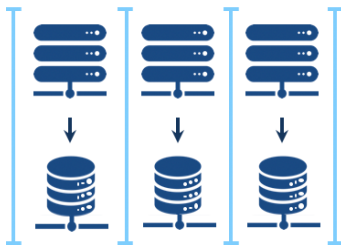
2

Firewalls & load balancers



3

Web servers
App servers
Storage



Generally Do Not Isolate –

Cost prohibitive to justify full isolation

Isolate if Practical –

- Firewalls and load balancers **bottleneck and fail**
- Ideally **each swim lane** will have its own firewalls and load balancers (in redundant pairs)

Always Isolate –

- Top of rack switches
- Compute
- Storage



Isolate faults – X axis

Fault isolation in X axis would mean

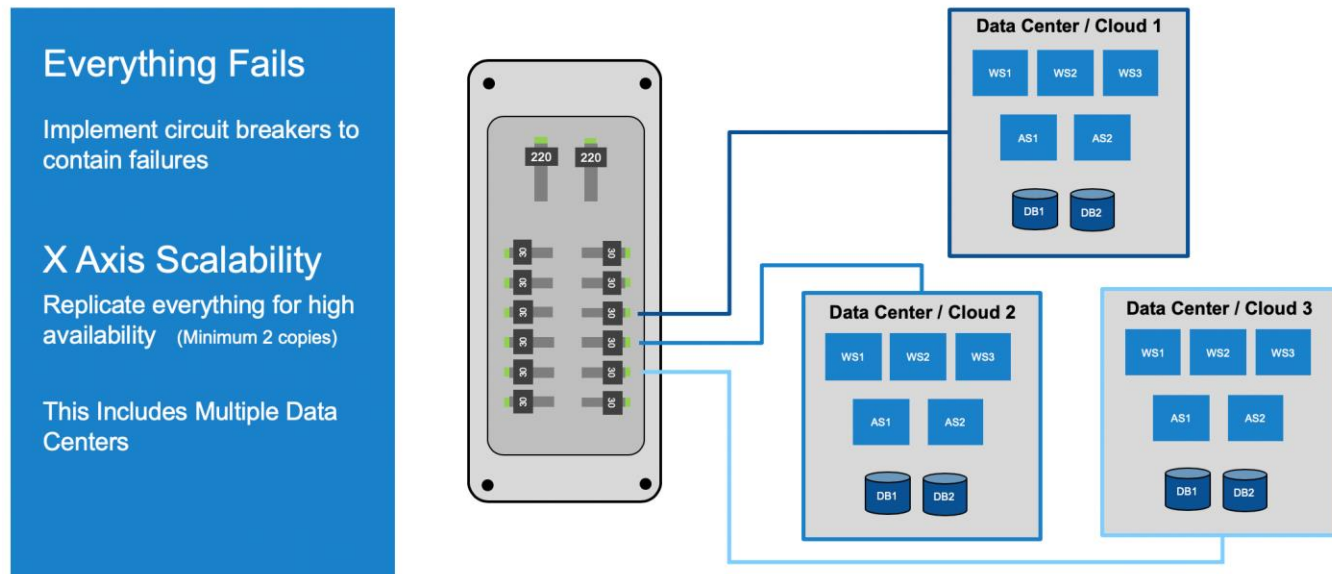
- replicating everything for high availability
- performing the replication asynchronously
- if possible, adopting an *eventually consistent* rather than a consistent solution



Isolate faults – X axis

For example, when a data center fails,

- the fault will be isolated to the one failed data center or multiple availability zones
- this solution is common with **traditional disaster recovery approaches**, though there might be better and less expensive solutions for recovering from a disaster



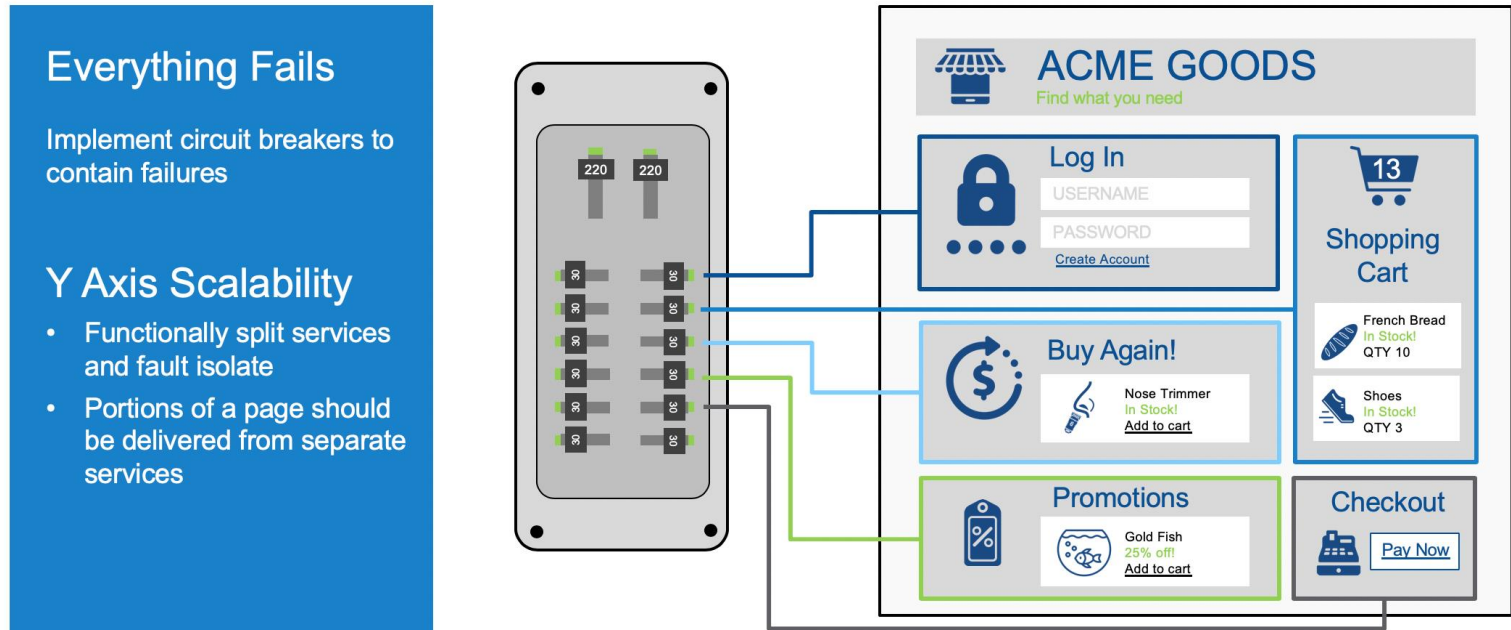
Isolate faults – Y axis

- Fault Isolation in the Y axis can be thought in terms of a separation of services, e.g., “login” and “shopping cart” in two separate swim-lanes
- Each lane having the Web and app servers as well as all data stores located within the swim-lane and answering only to systems within that swim-lane
- Each portion of a page is delivered from a separate service reducing the blast radius of a potential fault to its swim-lane



Isolate faults – Y axis

- Example of an **e-commerce site** showing different components of the page broken down into sections: login, buy again, promotions, shopping cart, and checkout
- Each component would reside within separate applications, hosted on different servers with properly isolated services. It can fail or succeed independently so that a failure of one component does not bring down the entire page/session



Isolate faults – Z axis

- Another approach is to perform a **separation of the customer base** or separation of the order numbers or product catalog
 - Assuming a function to perform this separation (like a modulus of id), such a split would be a Z axis swim-lane along customer, order number, or product ID lines
- If we are interested in the fastest possible response times to customers, we may even split along geographic boundaries with each pointing to the closest data center within that region
- Besides contributing to faster customer response times, these implementations can also help ensure that we are compliant with some ***data sovereignty laws*** (e.g., GDPR in EU or specific states in US)



Isolate faults – Z axis

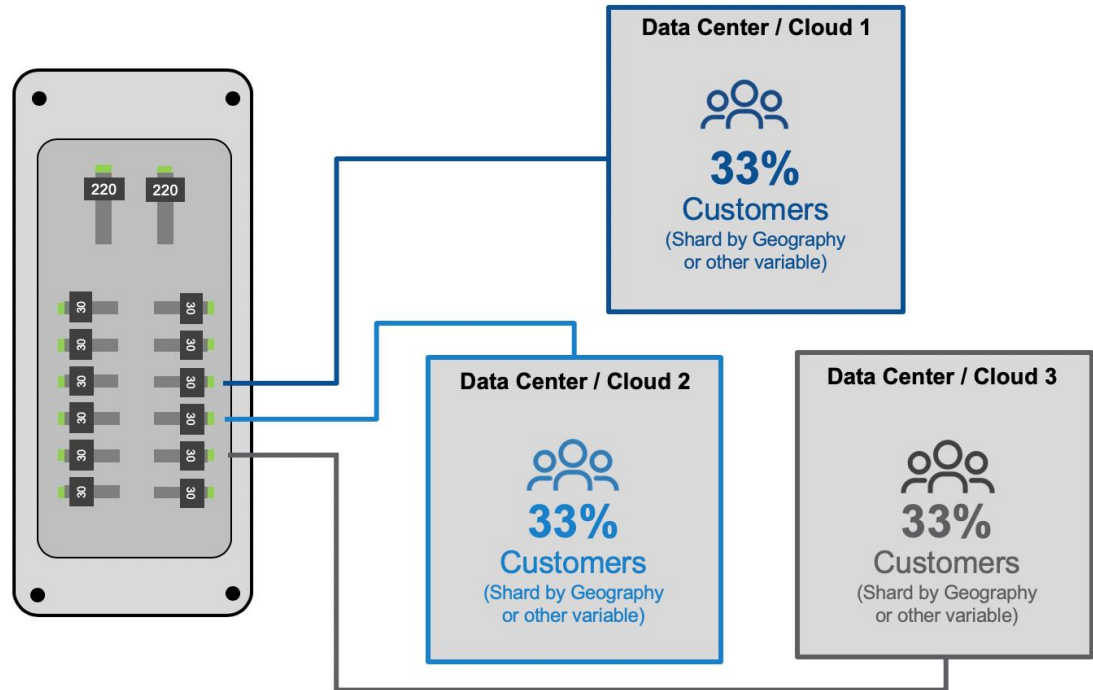
Everything Fails

Implement circuit breakers to contain failures

Z Axis Scalability

Split customers and assign them to different primary data centers for fault isolation

Data for all customers exists at all three data centers, should one fail, customers are rerouted



Combining the concepts of service and database separation into several fault isolative failure domains creates both a **scalable** and **highly available** platform through fault isolation



Design choices

The factor that mainly affects the complexity and requirements of a redundancy solution is the **failure rate of individual components** of a system:

- If individual components have **low failure rates**, a simple redundancy policy can be applied to provide high availability
- Components with **higher failure rates** require complex redundancy policies to mitigate any issues and provide guaranteed availability
- Another factor to consider when implementing a redundancy methodology is the **workload of the system**, as higher workloads cause system components to be constantly stressed, thus leading to faster component degradation

